

CS3002E Compiler Design

Monsoon 2025

Lecture #15

Bottom-Up Parsing - 4: LR(0) Parsing

Department of Computer Science & Engineering
NIT Calicut

LR(k) Parsing

LR(k)

- Left-to-right scanning of the input
- Rightmost derivation (in reverse)
- k symbols lookahead

More powerful than LL(k) parsing technique.

- LL(k): must predict the production based on k lookahead symbols.
- LR(k): reduction after seeing the entire right hand side of a production and k lookahead symbols.

LR(0) Parsing

- Parsing actions based on the stack contents.
- LR(0) grammars are too weak to be useful.

Computing canonical collection of sets of LR(0) items

Input: G , an augmented grammar with start symbol S'

Output: C , the canonical collection of sets of LR(0) items for G

$C = \text{Closure}(\{S' \rightarrow .S\})$

repeat

 for each set I in C

 for each item $A \rightarrow \alpha.X\beta$ in I

$J = \text{Goto}(I, X)$

$C = C \cup J$

until no new sets are added to C

LR(0) Parser Generation

- Compute $C = I_0, I_1, \dots, I_n$, the canonical collection of sets of LR(0) items.
- Each set I_i in C , represents a parser state i .
- The initial state of the parser is $I_0 = \text{Closure}(\{S' \rightarrow .S\})$
- For each state i
 - for each terminal a , if $\text{Goto}(I_i, a) = I_j$, set parser action as *shift a and move to state j* .
 - for each nonterminal A , if $\text{Goto}(I_i, A) = I_j$, set parser action as *move to state j* .
 - if I_i has an item, $A \rightarrow \alpha.$, $A \neq S'$, set parser action as *reduce α to A* .
 - if I_i has an item, $S' \rightarrow S.$, set parser action as *accept* for input symbol \$.

LR(0) Automaton

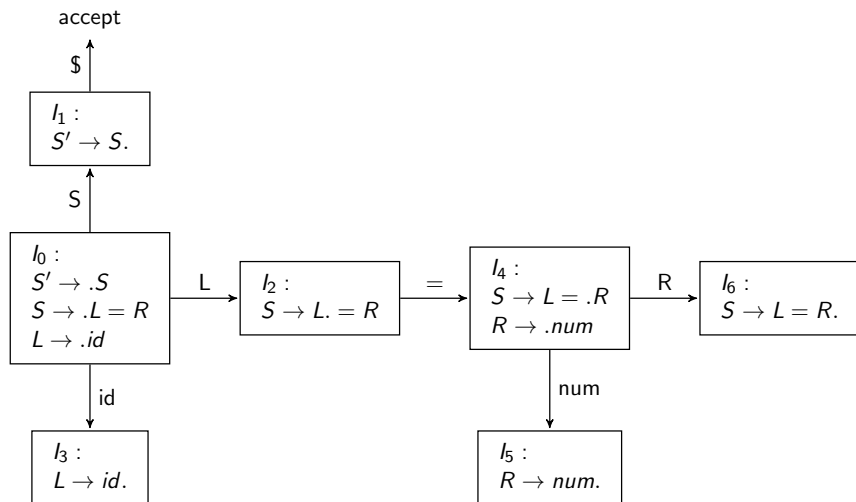
- Each state of the automaton represents a set of items in the canonical LR(0) collection.
- Initial state is $Closure(\{S' \rightarrow .S\})$
- State transitions are defined by the *Goto* function.
 - For a set of items I and a grammar symbol X , $Goto(I, X)$ is defined to be the closure of the set of all items $A \rightarrow \alpha X.\beta$ such that $A \rightarrow \alpha.X\beta$ is in I .

Example

$$S' \rightarrow S$$
$$S \rightarrow L = R$$
$$L \rightarrow id$$
$$R \rightarrow num$$

Initial state, $I_0 = Closure(\{S' \rightarrow .S\})$
 $= \{S' \rightarrow .S, S \rightarrow .L = R, L \rightarrow .id\}$

LR(0) Automaton



LR(0) Parsing Table

The parsing table has two parts - ACTION and GOTO.

For each state i , represented by set I_i

- If $[A \rightarrow \alpha.a\beta]$ is in I_i , and $Goto(I_i, a) = I_j$, $ACTION[i, a] = s_j$ (shift and goto state j)
- If $[A \rightarrow \alpha.]$ is in I_i and $A \neq S'$, $ACTION[i, a] = r_j$ (reduce using production numbered j , assuming $A \rightarrow \alpha$ is numbered j) for every terminal a . Also set $ACTION[i, \$]$ to r_j
- If $[S' \rightarrow S.]$ is in I_i , set $ACTION[i, \$]$ to *accept*
- If $Goto(I_i, A) = I_j$, set $GOTO(i, A)$ to j
- All undefined entries are marked *error* (blank entries)

LR(0) Parsing Table

0 : $S' \rightarrow S$ 1 : $S \rightarrow L = R$ 2 : $L \rightarrow id$ 3 : $R \rightarrow num$

State	id	=	num	\$	L	R	S
0	s3				2		1
1				acc			
2		s4					
3	r2	r2	r2	r2			
4			s5			6	
5	r3	r3	r3	r3			
6	r1	r1	r1	r1			

- s_i : shift and goto state i
- r_i : reduce using production numbered i
- blank entries are *error*

LR(0) Grammar

A grammar G belongs to the class LR(0), if an LR(0) parsing table can be built for G (without any conflicting entries in the parsing table).

Exercise

For each of the following augmented grammars, compute the canonical collection of sets of LR(0) items, draw the LR(0) automaton and check if the grammar is LR(0).

1. $E' \rightarrow E$
 $E \rightarrow E + T \mid T$
 $T \rightarrow id$

2. $S' \rightarrow S$
 $S \rightarrow id = R$
 $R \rightarrow id + id$
 $R \rightarrow id$

Exercise

Check if each of the below given grammars is LR(0)? If LR(0), construct LR(0) Parsing Table. If not, indicate the state(s) for which there is a conflict.

1. $E \rightarrow T + E \mid T$
 $T \rightarrow id$

2. $S \rightarrow (L) \mid x$
 $L \rightarrow S$
 $L \rightarrow L , S$

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.
- Appel A. W., Modern Compiler Implementation in Java, Cambridge University Press, 2000.

Further reading:

- ALSU chapter 4, section 4.6, Appel A. W, section 3.3